QueryAPIGate HOW-TO GUIDE 01 · START

Turn your first SQL query into a REST API

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=01-turn-your-first-sql-query-into-a-rest-api

---

Time: about 10 minutes. **You'll end up with:** a real HTTP endpoint, backed by your own SQL, with typed/validated parameters - callable with `curl`, from a browser, or from any HTTP client - and no controller, router or serialization code written by hand.

This guide uses the built-in example database (zero setup - no external database needed) so you can see the whole flow end to end first. Once you've done this once, connect your own database and repeat exactly the same steps against it.

What you'll need

- QueryAPIGate installed: `pip install "queryapigate[server]"` (or use the Docker image - see README.md).
- A terminal, and a browser.

Step 1: Start a server with real data to query

```shell
QUERYAPIGATE_API_KEY=demo-key queryapigate examples load
# also prints one scoped API key per example scenario - ignore those for now, you're using the admin key
QUERYAPIGATE_API_KEY=demo-key queryapigate serve
# Serving on http://127.0.0.1:5000
```

`examples load` installs a small generated database (`examples` - films, customers, rentals) and some worked saved queries you can look at later. Setting `QUERYAPIGATE_API_KEY` gives you an admin key (`demo-key`) to use through this guide - as soon as any API key exists, the server requires one for every request, so set it before you load anything.

Leave that terminal running. Open a second terminal (or your browser) for everything below.

Step 2: Open the admin UI and go to API Designer

Open **http://127.0.0.1:5000/console**. The first time, it asks for an API key - enter `demo-key`.

In the left sidebar, under **API**, click **API Designer**. This is the "write and run SQL, then save it as an endpoint" screen - distinct from **API Repository**, which lists endpoints you've already saved.

Step 3: Pick a connection and look at what's there

At the top of API Designer, the **Connection** dropdown lists every database connection this server knows about. Select **examples**.

Click the **Schema** tab on the right side of the screen. It lists every table (`film` , `customer` , `rental` , `payment` , `category` , …) - click one to see its columns. This is the same schema QueryAPIGate itself queries when it validates your SQL; no separate documentation to keep in sync.

Step 4: Write and run a query

In the SQL editor, replace whatever's there with:

```sql
SELECT film_id, title, rating, rental_rate
FROM film
WHERE rating = :rating
ORDER BY title
```

Click **Run**. You should see a real result grid - films with a `G` rating won't show yet, because `:rating` has no value bound to it. That's expected: `:rating` is a **bound parameter**, resolved at request time - the same mechanism documentation/API.md uses everywhere, never string-substituted into the SQL.

(If you want to see it actually run with a real value first: temporarily replace `:rating` with `'PG'` , click Run, see real rows come back, then put `:rating` back before continuing - this step isn't required, just reassuring the first time.)

Step 5: Save it as a new API

Click **Save as New API**, right below the editor. A form opens on the same screen - no navigation, your query stays exactly as you wrote it. Fill in:

FIELD	VALUE FOR THIS GUIDE
Filename	`films_by_rating`
Author	your name
Description	`Films with a given rating`
Collection	leave blank
Tags	leave blank
`query_parameters`	`{"rating": {"type": "str", "enum": ["G", "PG", "PG-13", "R", "NC-17"], "description": "MPAA rating"}}`

That last field is what turns `:rating` from "any text someone sends" into a validated, documented parameter - a request with a rating outside that list gets rejected with a clear error before your SQL ever runs. See How to use bound parameters safely for every available rule (`min` / `max` , `pattern` , `default` , `required` , and more).

Click **Save**. A toast confirms it, and **API Repository**'s query count in the sidebar increments - your query is now a real, versioned, saved endpoint.

Step 6: Call your new endpoint

```shell
curl 'http://127.0.0.1:5000/q/films_by_rating?rating=PG' -H 'X-API-Key: demo-key'
```

You should get back a real JSON array of films. Try an invalid value to see the validation you just configured actually enforced:

```shell
curl 'http://127.0.0.1:5000/q/films_by_rating?rating=NOT-A-RATING' -H 'X-API-Key: demo-key'
# {"error": "Invalid parameters: rating must be one of: G, PG, PG-13, R, NC-17",
#  "errors": {"rating": "must be one of: G, PG, PG-13, R, NC-17"}}
```

Other formats work the same way, no extra code:

```shell
curl 'http://127.0.0.1:5000/q/films_by_rating?rating=PG&format=csv' -H 'X-API-Key: demo-key'
curl 'http://127.0.0.1:5000/q/films_by_rating?rating=PG&format=yaml' -H 'X-API-Key: demo-key'
```

What just happened

You wrote one `SELECT` statement and ended up with an endpoint that has: typed/validated parameters, pagination (`?page` / `?page_size`), eight output formats (json/ndjson/csv/tsv/xml/yaml/xlsx/parquet), a version history (save it again under the same name and this becomes version 2, with version 1 still callable), and a run history visible in **API Repository**'s **History** tab for this query - all without writing a controller, a repository layer, or a serializer.

Nothing here required the admin key specifically except *saving* the query - once it's saved, you'd normally hand out a narrower, purpose-built key to whoever actually calls it. That's the next thing worth doing before this goes anywhere near production:

Next steps

- Connect your own database instead of the example one.
- Use bound parameters safely - every validation rule available, not just `enum`.
- Set up your first scoped API key - stop handing out the admin key.
- Cache a saved query's response - if `films_by_rating` gets called a lot and the data doesn't change every second.